

Práctica 3

- Cruz Chavez Isaac

Act 1: Se mantuvo un análisis lógico durante la respuesta d00 en cada una de las preguntas a lo largo del código

```
Problemas 2 Salida Consola De Depuración Puertos Terminal Run: Laboratorio + - [ ] [ ] ... [ ] [ ] X
```

```
+-----+-----+-----+-----+
| RESULTADO DEL BLOQUE 1
+-----+-----+-----+-----+
| expresion          | predijiste    | salio         |      |
+-----+-----+-----+-----+
| enteroSinValor     | 0             | 0             | ok    |
| decimalSinValor    | 0.0           | 0.0           | ok    |
| logicoSinValor      | false         | false         | ok    |
| (int) letraSinValor| 0             | 0             | ok    |
| textoSinValor       | null          | null          | ok    |
+-----+-----+-----+-----+
Sin fallos en este bloque.

=====
BLOQUE 2 - La division que miente
Fijate en el tipo de cada operando y en donde esta puesto el casting.
```

Act 2: Creí que dividir cero entre cero daría null. Sin embargo, en Java null se usa para referencias a objetos vacíos; en operaciones matemáticas con decimales (double), el resultado de una indeterminación aritmética se representa como NaN (Not a Number).

```
Problemas 2 Salida Consola De Depuración Puertos Terminal Run: Laboratorio + - [ ] [ ] ... [ ] [ ] X
```

```
+-----+-----+-----+-----+
| RESULTADO DEL BLOQUE 2
+-----+-----+-----+-----+
| expresion          | predijiste    | salio         |      |
+-----+-----+-----+-----+
| 7 / 2              | 3             | 3             | ok    |
| 7.0 / 2            | 3.5           | 3.5           | ok    |
| (double) 7 / 2     | 3.5           | 3.5           | ok    |
| (double) (7 / 2)   | 3             | 3.0           | ok    |
| 7.0 / 0            | infinity      | Infinity      | ok    |
| 0.0 / 0.0          | null          | NaN           | FALLO |
+-----+-----+-----+-----+
Fallaste 1. Anotalas: explicarlas es lo que se califica.
```

Act 3: En todas las operaciones de este bloque predije valores lógicos (true o false). Me confundí pensando que el símbolo % evaluaba una condición, pero en realidad es el operador matemático de módulo, el cual devuelve el residuo numérico de una división. Por eso `7 % 2` arrojó 1, y `10 % 5` arrojó 0

expresion	predijiste	salio	
<code>7 % 2</code>	false	1	FALLO
<code>10 % 5</code>	true	0	FALLO
<code>-7 % 2</code>	false	-1	FALLO
<code>7 % 2.5</code>	false	2.0	FALLO
<code>12345 % 10</code>	true	5	FALLO
<code>12345 / 10</code>	1234	1234	ok

Act 4: Predije que el sistema arrojaría los números 6 y 65. Mi error fue no considerar que el casting explícito a (char) obliga a Java a traducir el valor numérico a su símbolo correspondiente en la tabla ASCII; por lo tanto, el 65 se convierte en la letra A, y la suma de 'A' + 1 (que es 66) da como resultado B.

```
(char) ('A' + 1) = 65
(byte) 200 = -56
```

RESULTADO DEL BLOQUE 4			
expresion	predijiste	salio	
<code>(int) 3.9</code>	3	3	ok
<code>(int) -3.9</code>	-3	-3	ok
<code>(char) 65</code>	6	A	FALLO
<code>(int) 'A'</code>	65	65	ok
<code>'A' + 1</code>	66	66	ok
<code>(char) ('A' + 1)</code>	65	B	FALLO
<code>(byte) 200</code>	-56	-56	ok

Fallaste 2. Anotalas: explicarlas es lo que se califica.

Act 5: Predije que la igualdad sería true y que la suma daría 0.3 exacto. El programa arrojó false y un resultado de 0.300000~. Esto ocurre por un problema clásico de precisión de punto flotante: la arquitectura de la computadora no puede representar decimales con precisión infinita en binario, dejando minúsculos residuos en la memoria.

RESULTADO DEL BLOQUE 5			
expresion	predijiste	salio	
5 > 3	true	true	ok
5 == 5.0	true	true	ok
0.1 + 0.2 == 0.3	true	false	FALLO
true && false	false	false	ok
true false	true	true	ok
vidasRestantes > 0 && 10 / vi~	false	false	ok

Fallaste 1. Anotalas: explicarlas es lo que se califica.

Act 6: Pensé que el resultado sería la suma 15. Fallé porque Java evalúa la expresión de izquierda a derecha; como el primer elemento ("1") es un String, el compilador asume que toda la operación es una concatenación de texto, simplemente pegando los caracteres para formar "123" en lugar de sumar sus valores matemáticos.

RESULTADO DEL BLOQUE 6			
expresion	predijiste	salio	
0.1 + 0.2	0.3	0.300000~	FALLO
Integer.MAX_VALUE + 1	-2147483648	-2147483~	ok
5 + 3	8	8	ok
"5" + 3	53	53	ok
1 + 2 + "3"	33	33	ok
"1" + 2 + 3	15	123	FALLO

Act 7: Para esta última parte del laboratorio, el reto cambió de predecir a programar directamente las soluciones aplicando lo aprendido. Fue bastante útil porque me obligó a esquivar las "trampas" en las que caí durante los bloques anteriores. Por ejemplo, apliqué la división con un operando decimal (2.0) para forzar un resultado de punto flotante exacto, y comprobé que el casteo a (int) simplemente trunca los números sin redondear. Lo más interesante fue programar el "corto circuito" lógico con el operador &&, ya que pude comprobar cómo Java detiene la lectura de la instrucción, protegiendo al sistema de una división entre cero que habría hecho tronar el programa.

BLOQUE 7 · AHORA TU (aquí escribes código, no predicciones)

=====

7.1 = 2.5

7.2 = false

7.3 = 9

7.4 = true

7.5 = false